Lesson 11: Wave After Wave

Godot Engine

VIDEO GAME CREATION

This week: clearing a wave of enemies no longer means the game is over - it means the NEXT wave begins. Clear all 3, and that's when you really win.

Open your own [YourName]Game project from Lesson 10 - don't start a new one.

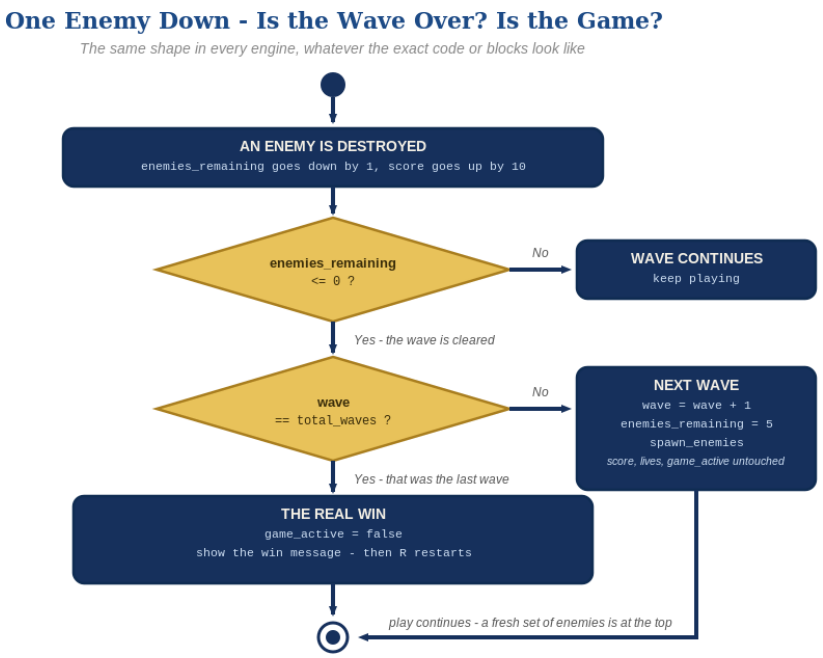
Tip: no new assets this week - waves reuse the spawn routine you already built in Lesson 10. This completes the Level 2 Complete milestone!

Setup

1. Open your [YourName]Game project from Lesson 10.
2. In your main scene, add a Label node (Godot's text-display node) and rename it WaveText. Place it near ScoreText and LivesText so all three are easy to see.

Level 1 - Minimal (MVP)

Clearing a wave's enemies spawns a fresh wave, up to 3 total. Clearing wave 3 triggers the real win. The current wave number is shown on screen.



Clearing a wave asks a second question. If it was not the last wave, the game spawns a fresh set of enemies and carries on. The same shape in every engine.

1. In GameManager's script, add var wave := 1 and var total_waves := 3 with the rest of the shared game state. GameManager owns every number the game shares.
2. In update_hud(), add a line that sets WaveText's text from wave and total_waves.
3. In start_game(), add wave = 1 alongside the other resets. Main's spawn_enemies() does not change at all this week - that is the payoff of Lesson 10.
4. In on_enemy_destroyed(), revise the win check into an if/else (code below): if this was the last wave, win as before. Otherwise advance wave, reset enemies_remaining and ask Main to spawn again. Notice the call is deferred: a wave is cleared in the middle of a collision, and Godot will not let you add new physics objects to the scene while it is still processing collisions. call_deferred waits until it is safe, the same idea as last week.
5. Run the scene and play through. Clearing wave 1 or 2 should bring a fresh set of five instead of ending the game. Clearing wave 3 should show WinText.

Highlighted lines are new this week; everything else is code you already have.

```
# GameManager's script (the new and revised members):
var enemies_remaining := 5
var lives := 3
var score := 0
var game_active := true
var wave := 1
var total_waves := 3

func start_game():
    for old_enemy in get_tree().get_nodes_in_group("enemies"):
        old_enemy.queue_free()
```

```
    enemies_remaining = 5
    lives = 3
    score = 0
    wave = 1
    game_active = true
    get_node("../WinText").hide()
    get_node("../LoseText").hide()
    update_hud()
    get_parent().spawn_enemies()

func update_hud():
    get_node("../ScoreText").text = "Score: " + str(score)
    get_node("../LivesText").text = "Lives: " + str(lives)
    get_node("../WaveText").text = "Wave: " + str(wave) + " of " +
str(total_waves)

func on_enemy_destroyed():
    enemies_remaining -= 1
    score += 10
    update_hud()
    if enemies_remaining <= 0:
        if wave >= total_waves:
            game_active = false
            get_node("../WinText").text = "You Win! Press R to Restart"
            get_node("../WinText").show()
        else:
            wave += 1
            enemies_remaining = 5
            update_hud()
            get_parent().call_deferred("spawn_enemies")

# Main scene's script: spawn_enemies() is unchanged.
```

Before You Test: Check Your Game State

Remember Lesson 10: a restart must put ALL game state back. This week you added new state, so check it too:

1. wave: confirm start_game() sets wave = 1, so a restart always begins at wave 1.
2. WaveText: it is updated by update_hud(), which start_game() already calls, so it resets itself. Anything extra you added for waves (a message, a Tween, a sound) needs its own reset inside start_game().

Level 2 - Complete (Alpha/Beta)

Wave transitions are seamless and correct every time - no double-spawns, no skipped waves, exactly 3 waves before the real win, and restarting always returns to wave 1. Nothing in the code needs to change for Level 2; it is about testing what you built.

- Play a full 3-wave game: confirm waves 1 and 2 each spawn a fresh set of five with no win message, and only wave 3 shows WinText.
- Let all 3 lives run out partway through wave 2 and confirm LoseText still appears, not a wave transition.
- Press R after a win or a loss and confirm wave is back to 1, along with score and lives, with exactly five enemies on screen.

Level 3 - Refined (Ready to Publish) - pick any that interest you

- The Stalemate is back - and now it can freeze the game at any of the three waves. If you fixed it in Lesson 10, check that your fix still works with waves. If you didn't, this may be the week it gets annoying enough to deal with. Find the cause and fix it in the way you, the Designer, think best. (Test it: let some enemies escape, destroy the rest, in wave 1, wave 2 and wave 3.)
- Visual: briefly show a "Wave 2!" message when a new wave spawns, using its own Label node (not WinText or LoseText, which stay reserved for the end of the round). Remember to hide it again in start_game().
- Structure: notice spawn_enemies() is now called from two places (start_game() and the else branch of on_enemy_destroyed()), but the enemy-creating code exists only once.

If Something's Wrong

- Wave 1 immediately shows WinText: confirm the if/else checks wave >= total_waves, and that the else branch is lined up with the inner if, not the outer one.
- New waves never spawn: confirm the call inside the else branch is spelled exactly like the function (spawn_enemies), including in the quotes.
- An error mentioning "flushing queries" in the Debugger: spawn_enemies() was called directly instead of with call_deferred.
- Old enemies pile up on restart: confirm enemy.tscn's root node is still in the "enemies" group (Lesson 10 setup).
- A wave never ends after you let some enemies escape - no next wave, no win, no Game Over: that is the Stalemate from Lesson 10 (Level 3), not a typo in your code. Escaped enemies cost a life but never count toward clearing the wave.

Want More?

Student Resource List > Godot section, or docs.godotengine.org